Operating System Reference Notes

Unit 1: Introduction, Processes, and CPU Scheduling

1. Operating System Overview

An **Operating System (OS)** is a program that acts as an intermediary between a user of a computer and the computer hardware.

- **Goals:** Execute user programs, make the computer system convenient to use, and use computer hardware in an efficient manner.
- **User View:** Focuses on ease of use and performance.
- **System View:** The OS is a resource allocator (CPU time, memory, I/O) and a control program that manages the execution of programs to prevent errors.

Types of Systems:

- **Batch Systems:** Jobs with similar needs are batched together and executed as a group.
- **Multi-programmed Systems:** Several jobs are kept in main memory at once; the OS picks one and begins to execute. It increases CPU utilization.
- **Time-Sharing Systems:** An extension of multi-programming where the CPU executes multiple jobs by switching among them so frequently that users can interact with each program while it is running.
- **Real-Time Systems:** Used when rigid time requirements have been placed on the operation of a processor (e.g., medical imaging, weapon systems).

2. Process Concept & Scheduling

A **Process** is a program in execution.

- **Process State Diagram:** A process moves through states: **New, Ready, Running, Waiting, and Terminated**.
- **Process Control Block (PCB):** A data structure containing information about a process: Process State, Program Counter, CPU Registers, Memory Management info, and I/O status.
- **Scheduling Queues:** Job Queue (all processes), Ready Queue (processes in main memory ready to run), and Device Queues.

- **Schedulers:** * *Long-term:* Selects which processes should be brought into the ready queue.
    - *Short-term:* Selects which process should be executed next by the CPU.
- **Interprocess Communication (IPC):** Mechanisms to allow processes to communicate (Shared Memory and Message Passing).

3. CPU Scheduling Algorithms

Scheduling can be **Preemptive** (OS can interrupt a running process) or **Non-Preemptive** (Process holds CPU until it terminates or waits).

Algorithms:

1. **FCFS (First-Come, First-Served):** Simplest, but suffers from the "Convoy Effect."
2. **SJF (Shortest Job First):** Optimal for minimum average waiting time.
3. **Priority Scheduling:** CPU allocated to the process with the highest priority.
4. **Round Robin (RR):** Each process gets a small unit of CPU time (time quantum). Designed for time-sharing.

4. **The PCB (The Heart of a Process):** Every process is represented in the OS by a **Process Control Block**.

1. **Process State:** (New, Ready, Running, Waiting, Terminated).
2. **Program Counter:** Indicates the address of the next instruction.
3. **CPU Registers:** Includes accumulators, index registers, and general-purpose registers.
4. **CPU-Scheduling Information:** Includes priority and pointers to scheduling queues.
5. **Memory-management Information:** Value of base and limit registers, page tables.
6. **Accounting Information:** Amount of CPU used, time limits, account numbers.

Unit 2: Process Synchronization and Deadlocks

1. Process Synchronization

When multiple processes access shared data, a **Race Condition** may occur.

- **Critical Section Problem:** A segment of code where shared resources are accessed.
    - *Solution Requirements:* Mutual Exclusion, Progress, and Bounded Waiting.
- **Bakery Algorithm:** A solution for n processes to enter the critical section safely.

- **Semaphores:** Integer variables used for signaling.
    - *Wait (P)* and *Signal (V)* operations.
- **Classic Problems:** * **Bounded-Buffer:** Producer and Consumer sharing a fixed-size buffer.
    - **Readers-Writers:** Multiple readers allowed, but only one writer at a time.

2. Deadlocks

A situation where a set of processes are blocked because each is holding a resource and waiting for another resource held by another process.

- **Deadlock Characterization (Four Conditions):** Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait.
- **Handling Methods:**
    1. **Prevention:** Ensure at least one of the four conditions never holds.
    2. **Avoidance:** Uses the **Banker's Algorithm** to ensure the system stays in a "Safe State."
    3. **Detection & Recovery:** Periodically check for deadlocks and terminate processes or preempt resources to break the cycle.
- **Unit 3: Memory Management and File Systems**
- **1. Paging and Segmentation**
- **Paging:** Logical memory is divided into **Pages**; Physical memory into **Frames**. The **Page Table** maps pages to frames.
- **Segmentation:** Memory is divided into segments based on the programmer's view (Code, Stack, Library, Main). A **Segment Table** contains the base address and the limit (length) of each segment.
- **2. Virtual Memory: Demand Paging**
- The OS does not load the entire program into RAM. It uses **Demand Paging**.
- CPU refers to a page.
- Check the Page Table. If the "Valid/Invalid" bit is 'I', it is a **Page Fault**.
- OS finds the page on the disk.
- Swap the page into a free frame in RAM.
- Update the Page Table to 'V'.

- Restart the instruction.
- **3. File Allocation Methods**
- **Contiguous:** File occupies a set of consecutive blocks. **Pro:** Great for sequential access. **Con:** External fragmentation.
- **Linked:** Each block contains a pointer to the next block. **Pro:** No fragmentation. **Con:** Slow (random access is impossible).
- **Indexed:** An index block stores pointers to all data blocks. **Pro:** Supports direct access without fragmentation.

Unit 4: Unix and Shell Programming

1. Unix Architecture Layers

1. **Hardware:** The physical machine.
2. **Kernel:** Interacts with hardware. Manages memory, files, and scheduling.
3. **Shell:** The interface that interprets your commands and passes them to the kernel.
4. **Applications:** User-level programs like ls, grep, pwd.

2. Shell Scripting Control Structures

- **If Statement:**

  Bash

  ```
  if [ condition ]
  then
    # commands
  fi
  ```

- **Case Statement:** (Equivalent to Switch-Case)

  Bash

  ```
  case $variable in
  pattern1) commands ;;
  pattern2) commands ;;
  *) default_commands ;;
  esac
  ```

- **Loops (For/While/Until):**

- while: Runs as long as the condition is **true**.
- until: Runs as long as the condition is **false**.

3. Key Shell Commands for Exams

- expr: Used for arithmetic operations (e.g., sum=`expr $a + $b`).
- echo: Displays text or variables.
- read: Takes input from the user.
- test: Used to compare numbers (-eq, -ne) or check files (-d, -f).